

**Project**  
**REF: Resource Evaluation and Feedback DSL**  
**Project Assignment P1: Modeling**  
**ENORM**

Master in Informatics Engineering - 2025/2026

Porto, February 22, 2026

Alexandre Bragança (ATB),

Rafael Gonçalves (RCC)

Version 5, 2026-02-19



## REVISION HISTORY

| Revision | Date       | Author(s) | Description                                |
|----------|------------|-----------|--------------------------------------------|
| 1        | 2026-02-11 | atb       | Initial version                            |
| 2        | 2026-02-13 | atb       | First complete version                     |
| 3        | 2026-02-16 | atb       | Add Reddit picture and minor other updates |
| 4        | 2026-02-19 | atb       | Minor fix                                  |
| 5        | 2026-02-19 | atb       | Updates submission of P1 to 18 of April    |



# CONTENTS

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Context</b>                                            | <b>1</b>  |
| 1.1      | The Problem . . . . .                                     | 1         |
| 1.2      | REF: A DSL for Resource Evaluation and Feedback . . . . . | 2         |
| 1.2.1    | Simple, Intuitive and Easier to Learn . . . . .           | 2         |
| 1.2.2    | Strongly Typed . . . . .                                  | 3         |
| 1.2.3    | Complete . . . . .                                        | 4         |
| 1.2.4    | Flexible and Extensible . . . . .                         | 4         |
| 1.2.5    | Evolvable . . . . .                                       | 4         |
| 1.3      | DSL Engineering . . . . .                                 | 4         |
| 1.3.1    | Domain Engineering . . . . .                              | 5         |
| 1.3.2    | Application Engineering . . . . .                         | 5         |
| <b>2</b> | <b>Organization and Process</b>                           | <b>9</b>  |
| 2.1      | Teams and Tools . . . . .                                 | 9         |
| 2.2      | Repositories . . . . .                                    | 10        |
| 2.3      | Reports . . . . .                                         | 11        |
| 2.4      | Process . . . . .                                         | 11        |
| 2.5      | Submission . . . . .                                      | 14        |
| <b>3</b> | <b>Domain Engineering</b>                                 | <b>15</b> |
| 3.1      | Domain Knowledge . . . . .                                | 15        |
| 3.2      | Metamodel . . . . .                                       | 15        |
| 3.3      | Metamodel Constrains . . . . .                            | 16        |
| 3.4      | Model Transformations . . . . .                           | 17        |
| 3.5      | Model Visualizations/Projections . . . . .                | 17        |
| <b>4</b> | <b>Terms and conditions</b>                               | <b>21</b> |
| 4.1      | Submission . . . . .                                      | 21        |
| 4.2      | Report . . . . .                                          | 21        |

|       |                            |    |
|-------|----------------------------|----|
| 4.3   | Deadline . . . . .         | 21 |
| 4.4   | Feedback . . . . .         | 21 |
| 4.5   | Evaluation . . . . .       | 21 |
| 4.6   | Grading . . . . .          | 22 |
| 4.6.1 | Grading Criteria . . . . . | 22 |
| 4.7   | Code of Conduct . . . . .  | 23 |

## LIST OF FIGURES

|     |                                                                                                                                                                                                                              |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | A simple textual syntax (from <a href="http://www.wcse.org/WCSE_2017/041.pdf">http://www.wcse.org/WCSE_2017/041.pdf</a>                                                                                                      | 3  |
| 1.2 | An Example of Graphical syntax based on BPMN (from <a href="https://pdfs.semanticscholar.org/ca5/ee06b4ec40099ae5ee7f803ef10a59b2fe25.pdf">https://pdfs.semanticscholar.org/ca5/ee06b4ec40099ae5ee7f803ef10a59b2fe25.pdf</a> | 3  |
| 1.3 | Domain Engineering.                                                                                                                                                                                                          | 5  |
| 1.4 | Application Engineering.                                                                                                                                                                                                     | 6  |
| 3.1 | Reddit Automations.                                                                                                                                                                                                          | 16 |
| 3.2 | An example diagram generated by Plantuml/dot                                                                                                                                                                                 | 19 |





## LISTINGS

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 3.1 | Excerpt of possible textual projection for a REF model . . . . .       | 18 |
| 3.2 | Example of Plantuml/dot source code for generating a diagram . . . . . | 18 |



Chapter

1

## CONTEXT

This document describes the first part (P1) of the ENORM project, which overall goal is the exploratory development of tools in the context of a Software Product Line. Specially, the aim is to develop a domain-specific language and some other supporting tools. This chapter presents an overview of the ENORM project, focusing on part P1 of the project. Chapter 2 presents some guidelines on the organization and procedures to be used. Chapter 3 presents details of the development of the core assets of the product line. Finally, Chapter 4 presents the terms and conditions of the project.

### 1.1 The Problem

The ENORM project considers the simplified case of a software company that is specialized in the development of web based solutions. **The company has developed several web solutions and, through the years it has identify some commonalities in those solutions.** Resulting from this constact it has decided to explore those **using a software product line (SPL) approach.** In the context of the new SPL approach, it will also explore the use of a DSL. **The ENORM project regards the exploration of this DSL.**

The great majority of the **web solutions developed by the company include some sort of user evaluation or feedback features.** For instance, in e-commerce applications it is very common to allow users to evaluate the products they buy. Similarly, in web applications were users post content (such as videos or photos) it is usual to give users the possibility to comment on such posts.

One of the technical aspects of **these solutions is that they rely on a REST architecture,** and the backend is composed of several REST API based microservices that are very similar. The first approach of the company was to use a DSL to help developers create these APIs. As such, the company developed a DSL that helped developers generate a significant part of the

code<sup>1</sup>.

The results of this initial DSL were promising. However, the DSL is technical and aimed at software engineers. The company now wishes **to explore a more high level DSL, one that can be used by subject matter experts (SME)**. The ENORM project is this next phase of the exploration of this approach. **The main objectives are to build a MVP of a DSL that:**

- can generate at least 90% of the code required to implement the backend microservices that support the users feedback and evaluation of the web applications produced by the company;
- can be used by subject matter experts as they describe/specify their needs in this context;
- supports both textual and graphical syntaxes;

The next section will present more specific aims of the project.

## 1.2 REF: A DSL for Resource Evaluation and Feedback

**The goal for the project is to produce a DSL called REF (Resource Evaluation and Feedback).** As described before, this DSL is inspired by previous work of the company developing a DSL for REST API creation. The previous DSL targeted the software engineers of the company, reducing the time they spent coding very similar REST web services. This experience was successful, and now the company wants evolve even further the usage of DSLs to new users. This time it is targeting the subject matter experts (SME). These are usually professionals from the clients of the company that know very well the business domain, but do not have the technical skills for developing the solutions, they are not software engineers. **The aim is now to create a new DSL that will be used by these SME to specify their domain applications.** Based on these application specifications done with the new DSL, it should be possible to generate the majority of the code. The software engineers that previously developed the solutions will be trained to be able create and maintain the DSL as well as coding the possible 10% of code that will not be generated by the DSL.

The following sections detail some main characteristics that the DSL should support.

### 1.2.1 Simple, Intuitive and Easier to Learn

The DSL should be simple and easier to learn. It should be as much intuitive as possible for the subject matter expert.

The company made some research about DSL syntaxes and found some examples for other domains. For instance, Figure 1.1 and Figure 1.2 present examples of textual and graphical syntaxes of DSLs aimed at modeling Extract-Transform-Load (ETL) processes.

---

<sup>1</sup>A solution similar to the one available in <https://github.com/HachimHassani/Rest-DSL>

The textual syntax of the new DSL should be simple and easier to learn. **It should be, as much as possible, close to how a person would describe the application in natural language.**

Figure 1.1: A simple textual syntax (from [http://www.wcse.org/WCSE\\_2017/041.pdf](http://www.wcse.org/WCSE_2017/041.pdf))

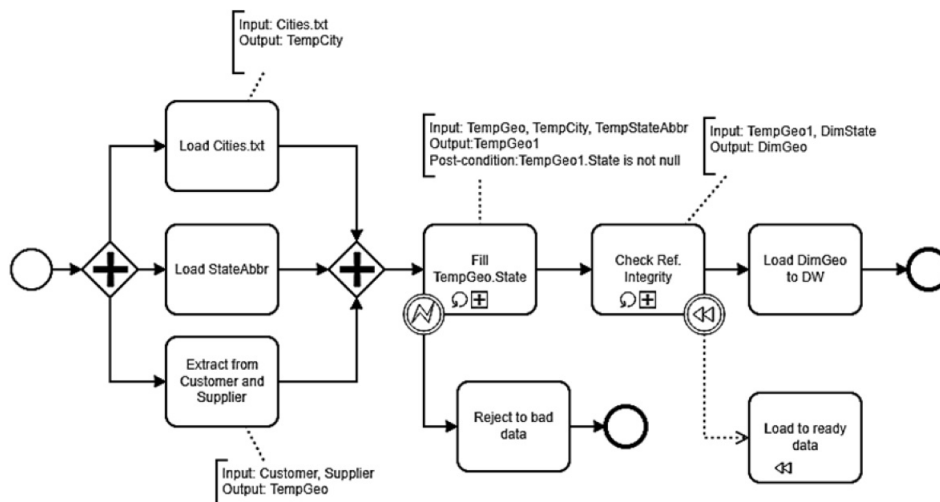
```

jobname:loadFileandDataDraft
author: Sunisa Junsawang
version:1.0
description:loadFileandDataDraft
step: FileLoadtoTableA
type: loadfile
source: input_file/test.file
format: header=yes , footer=no , delimiter=pipe , quote=no
target: tableA (append)
end
step: deletetoTableC
type: delete
target: tableC
condition: att3 > '2000'
end

```

Regarding the graphical notation, it should also be simple and **use intuitive graphical elements**. It should avoid cluttering the diagrams with too much detail. **A common way to avoid cluttering is to display detail as text in property windows.**

Figure 1.2: An Example of Graphical syntax based on BPMN (from <https://pdfs.semanticscholar.org/caf5/ee06b4ec40099ae5ee7f803ef10a59b2fe25.pdf>)



### 1.2.2 Strongly Typed

The DSL should be strongly typed, in order to help its users (SME) to avoid specification errors when writing DSL programs. Basically, the idea is that **when the user ends the creation of**

**a DSL program, that program/specification should always be a valid specification so that the generated code is also always valid.**

### 1.2.3 Complete

A domain usually includes not only structural but also behavioral elements. They take form as rules or processes that apply to the domain. We expect that this also applies to this REF domain. Therefore, it is expected that **the new DSL includes support to model both structural and behavioral aspects of the domain.**

### 1.2.4 Flexible and Extensible

Although the DSL should be simple and provide the most common features for the resource evaluation and feedback domain, it is possible that these functionalities do not cover all the cases. Therefore, the DSL should be extensible and flexible by, for instance, **providing an extensibility mechanism that the SME can use to add new operators/functions in the DSL that are then implemented by a software engineer in the base programming language (e.g., java).**

### 1.2.5 Evolvable

The DSL should be designed to support its evolution. When working with DSLs their evolution is a mandatory feature. If a DSL is to be successfully used it must be designed up-front thinking that it will, sooner or later, require changes. Therefore, **it is required to prepare the DSL with the necessary tools to support the migration of DSL programs to new versions of the DSL.**

## 1.3 DSL Engineering

The development process is divided in two levels: **domain engineering** and **application engineering**. These levels of engineering, although connected, are usually performed by very different roles. Domain engineering requires someone that is expert in building domain tools, such as domain-specific languages. Application engineering requires another role, someone which will use the previous domain tools (e.g., DSL) to build applications in the domain. This requires knowledge in the domain, but not on building domain tools (e.g., building DSLs), usually this is the role of the subject matter expert. For the ENORM project, you should have to play the two roles.

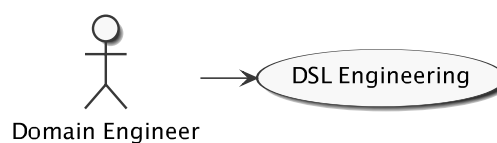
### 1.3.1 Domain Engineering

**Domain engineering** is responsible for developing reusable artifacts, i.e., domain-specific artifacts that are reused when building domain applications. In this particular case, the goal is to develop a **domain-specific language and supporting processes/tools** (e.g., model validations, model transformations and code generation) for building applications (in this specific case, backend services of the applications) (see Figure 1.3). The artifacts that result from domain engineering are then used at the level of **application engineering** that, obviously, in this case, will use the REF DSL to model and generate backend services that support resource evaluation and feedback processes of applications for the domain (see Figure 1.4).

**For this part of the project (P1), the major goal is to design the metamodel of the new REF language. This includes not only the structure of concepts (its properties and relations) but also model validations and model transformations (refactorings).** To construct the metamodel, domain knowledge must be extracted from studying documents and talking to the subject matter experts (in this case, teachers will play this role). With this knowledge, a design for the metamodel of the new language must be proposed that tries to comply with the previous presented aims and tries to support the specific requirements presented in section 3. You should also comply with the instructions of the laboratory class teacher that, in some way, will play the roles of subject matter expert and product owner.

**For this part of the project (P1), it is not necessary to develop the full DSL!** You are only required to build the parts described previously. It is not required to design and build the concrete syntax of the DSL, only if the tool you use requires it. If so, you may develop a "raw" syntax only to support the aims of P1. Therefore, **for P1, you are not required to design the concrete syntax of the DSL and you are not required to generate code.** It is, however, also required to develop the tasks described in the next section (Section 1.3.2), that are related to application engineering.

Figure 1.3: Domain Engineering.

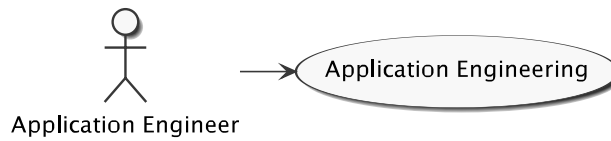


### 1.3.2 Application Engineering

Application engineers reuse the tools developed in domain engineering (e.g., DSLs) to create domain applications, as depicted in Figure 1.4. **When the reusable tool is a DSL the application engineer role can potentially be played by a subject matter expert.**

Since the goal of this project is to evaluate the feasibility of this approach in an MVP, it was decided to replicate three well-known cases of REF. Therefore, you should demonstrate the

Figure 1.4: Application Engineering.



solution with the following cases, that are briefly described:

- **Case 1: Youtube:** In this case the evaluated resources are user videos. Users can comment other users' videos and 'rate' them using thumbs up or down. Also, comments may start threads. Comments are usually sorted by date or "value". Users may report comments that violate community rules;
- **Case 2: Amazon:** In this case the evaluated resources are the products that users buy. Users can evaluate products. Usually users that evaluate products are also buyers, but this rule is not mandatory. Users classify the products using a five star grading scale. The evaluation includes text but may also include photos and videos. There is also a verification process for evaluations that can be manual or automatic;
- **Case 3: Reddit:** In this case the resource is similar to a discussion forum that has a specific topic. Users can post in the forum. There are different types of posts. When users reply to posts they create discussion threads. Users can evaluate posts and comments by "upvoting" and "downvoting" on them. Users may report on posts and comments. Each topic has its own rules and moderators. The platform supports a mechanism to automate some related processes.

**The developed DSL must support all these three scenarios.** It is up to you to research the specific requirements and features of these 3 scenarios. The previous description is only a high level overview of features.

The company is expecting that the designed DSL includes/supports the following requirements:

- **Structure:** it should be possible to **model the diverse properties of resources**;
- **Authorization:** it should be possible to **model the diverse users/actors that interact with the application and what operations can they perform**;
- **Behaviour:** it should be possible to **model the diverse processes related to the actions on the resource**, such as moderation processes and automatic validations. Reddit seems particularly powerful in this regard, and the company is expecting support for something similar to Reddit's **Automations**.



Remember that the goal of this MVP is to initially support at least these scenarios. It does not have to mimic exactly the three applications, but it must support similar applications. Also, the idea is that **the DSL should support its evolution, as new scenarios are identified and the DSL is updated to support them.**

**The modeled scenarios should provide sufficient data to demonstrate and test the major features of the DSL regarding its structure, validations and refactorings.**



## ORGANIZATION AND PROCESS

This ENORM project is divided in three parts or components:

- **P1 Modeling**

- This is the first part of the development of the DSL, focusing on modeling
- It starts at week 2 and its planned to end by the middle of the semester (at the end of week 7)
- It is the part that is described in this document
- This is a team component with individual activities and individual grading

- **P2 DSL and Code Generation**

- This is the second part of the development of the DSL, focusing on the DSL and code generation
- It is planned to start at the beginning of week 8 and ending at the end week 12
- It will be described in another document
- This is a team component with individual activities and individual grading

This document only describes the requirements for **P1 Modeling**.

### 2.1 Teams and Tools

Students should form teams of three elements (exceptionally teams can have less members, but only with the teacher's acceptance).

Each member of the team is responsible for the development of the solution using a specific tool. You should **check the technical requirements for the installation and usage of the tool**, e.g., some tools may have specific requirements.

Each team member should select a unique tool from the following ones. All members of the same team should select different tools.

- **MPS - Meta Programming System**

- Site: <https://www.jetbrains.com/mps/>
- Documentation: <https://www.jetbrains.com/help/mps/mps-user-s-guide.html>
- Tutorial video: <https://youtu.be/T1RErEvgRc>

- **Xtext**

- Site: <https://www.eclipse.org/Xtext/>
- Documentation: <https://www.eclipse.org/Xtext/documentation/index.html>
- Demonstration/Tutorial video: <https://vimeo.com/138873446>

- **Sirius**

- Site: <https://www.eclipse.org/sirius/>
- Documentation: <https://www.eclipse.org/sirius/doc/>
- Advanced documentation available at <https://wiki.eclipse.org/Sirius/Tutorials/AdvancedTutorial>
- Tutorial videos:
  - \* Part 1 <https://youtu.be/uKBGVwmRzi0>
  - \* Part 2 <https://youtu.be/3rnBmG4DN4w>
  - \* Part 3 <https://youtu.be/h0U62wf3bYs>
  - \* Part 4 [https://youtu.be/M1RdRsLic\\_k](https://youtu.be/M1RdRsLic_k)
  - \* Part 5 <https://youtu.be/iHtmL-PKT8c>

**Attention for Sirius and Xtext Tools:** These tools are based on EMF. They can use a metamodel created as an ecore metamodel, as described in the following tutorial <https://www.vogella.com/tutorials/EclipseEMF/article.html>. Therefore, in the case of these tools, you may follow this tutorial to learn how to create ecore metamodels (or follow the demonstrations from the lectures)

The students may select other tools but only with the teacher's acceptance.

## 2.2 Repositories

Each team will have access to a team repository that should be used to develop the two team components: P1 Modeling and P2 DSL and Code generation. Further instructions will be available in the team repository.

## 2.3 Reports

For each part of the project (P1 and P2) there will be a report to be produced. All reports are produced inside the repositories using Markdown. Further instructions related to the reports will be available in the repositories.

For this part of the project (P1), the report is included in the team repository. However, the report is further divided in two parts: a team part and individual parts. Team and Individual parts are to be developed and reported according to instructions in Section 2.4 and also specific teacher's indications.

## 2.4 Process

The proposed process for P1 Modeling is structured in the following activities. Usually they should be developed in a sequential way, for instance, one in each week. But this organisation is not mandatory, the team should organize the work as best fits their needs.

- **Activity 1: Team formation and Tool Research**

- Team tasks:
  - \* Select members of the team
  - \* Study the proposed tools
  - \* Add to the component P1 a report with a briefly description of each tool

- **Activity 2: Specific Tool Setup**

- Individual tasks:
  - \* Each student should select his/her tool
  - \* Each student should install his selected tool and try to experiment with the tools, following the tutorial videos presented previously
  - \* Add to the team repository a report (individual section), a description of the tool and instructions on how to install and setup the tool

- **Activity 3: Research Existing Knowledge**

- Individual tasks:
  - \* Each student should study and explore the domain knowledge of one of the scenarios described in Section 1.3.2
  - \* Each student should produce a domain model for one application scenario
  - \* See Chapter 3 for further instructions and requirements

- \* Add to the report (individual section) the domain knowledge obtained from analyzing the languages (including the domain model)

- **Activity 4: Design Domain Metamodel**

- Team tasks:
  - \* Design a metamodel that supports the new DSL (it should be supported and justified using the knowledge from the previous activity)
  - \* Use a formal notation (e.g., class diagram or similar)
  - \* Add to the report (team section) the design of the metamodel
- Individual tasks:
  - \* Implement the designed metamodel in the selected tool
    - **Note:** Xtext and Sirius are based on EMF, therefore it is possible to use EMF/Ecore to implement the metamodel
    - **MPS:** For MPS you should use **Structure**
  - \* Add to the report (individual section) a description of the implementation of the metamodel

- **Activity 5: Design Metamodel Constraints and Transformations**

- Team tasks:
  - \* Further analyse the domain applications to add constraints to the metamodel
  - \* Use a formal notation (e.g., OCL) or, at least, pseudo-code
  - \* Include for each constraint, when possible, a proposal of a refactoring to correct the model error
  - \* Add to the report (team section) the design of the metamodel constraints and refactorings
- Individual tasks:
  - \* Implement the designed constraints and refactoring in the selected tool
    - **Xtext** Use the wizard "Xtext Project from Existing Ecore Models" to create an Xtext DSL that is based on your previous metamodel. This will automatically generate a default textual representation for the DSL. Use this default representation. In part 2 of the project you will focus on the textual representation (not now). You can use Xtext's **Validation Rules** and **Quick Fixes** for this activity.
    - **Sirius** To be able to use the specific features of Sirius you need to implement a basic/minimum graphical representation for your DSL. You can use Sirius' **Validation Rules** and **Quick Fixes** for this activity.

- **MPS** MPS automatically provides a default editor for the DSL. Therefore you do not need to implement one for this part of the project. You can use MPS' **Checking Rules** and **Quick Fixes** for this activity.
- \* Add to the report (individual section) a description of the implementation of the constraints and refactorings

- **Activity 6: Design Model Projections/Visualizations**

- Team tasks:

- \* Research for textual and graphical notations for building model projections/visualizations for the new DSL
    - \* Do not forget that graphical projections/visualizations will be supported by plantuml (<https://plantuml.com>)
    - \* Add to the report (team section) a description on how each metamodel element will be represented (graphically and textually)

- Individual tasks:

- \* Implement the designed visualizations in the selected tool (or by any other means when not possible to integrate visualizations/projections with the tool)
      - **Xtext** You can use the generation features of Xtext to generate the PlantUML file (using **Code Generation**). Alternatively, **Custom Validation** and **Quick fixes** could be used to execute Java code that generates the PlantUML file (although this is not the intended design usage for quick fixes). A similar approach can be used to generate a textual projection for DSL models.
      - **Sirius** Usually for Sirius a tool named Acceleo is used for code/text generation. This tool is very powerful and will be mandatory in part 2 of the project. For now you could use a simpler solution, the feature "External Java Action Call" (see the advanced tutorial of Sirius), to execute Java code that generates the PlantUML file. A similar approach can be used to generate a textual projection for DSL models.
      - **MPS** You may implement a **Behavior** in MPS with the code that generates the PlantUML File. Then you can use an **Intention** to provide a popup menu option that executes the Behavior upon user request. Alternatively you may use one the generation features of MPS: **TextGen**. Similar approaches can be used to generate a textual projection for DSL models.
    - \* Add to the report (individual section) a description of the implementation of the visualizations/projections

- **Activity 7: Application Engineering**

- Team tasks:
  - \* Design models for the scenarios presented in Section 1.3.2
  - \* For the design of these models you may use the same notation as in UML object diagrams
  - \* Add to the report (team section) the design of the models
- Individual tasks:
  - \* Implement the designed models in the selected tool
  - \* Check constraints and refactoring
  - \* For all the models include errors that are detected by constraints and include refactoring to correct them
  - \* Generate visualizations/projections for all the models
  - \* Add to the report (individual section) a description of the previous individual tasks

**Note:** Application engineering is about using the tools from domain engineering to build new applications in the domain. For this first part of the project, no real programming or generation of applications is required for Application Engineering. Only the specific goals presented for Activity 7.

## 2.5 Submission

By the **end of week 7** (specific date to be announced) all members of the team should submit a copy of the team repository in Moodle (link to be announced).



To help the fulfillment of all the activities presented in Section 2.4 some hints and guidelines are presented in this Chapter.

### 3.1 Domain Knowledge

Domain knowledge should be gathered essentially by researching the applications presented in the scenarios that are to be supported: Youtube, Amazon and Reddit. For each application the domain knowledge should be captured using, at least, a domain model and some complementary text to justify options and include invariants and rules.

You should start by exploring and studying the applications with the focus on the specific theme of the DSL, i.e., how these applications handle user evaluation and feedback. Take the point of view of "someone" that is describing the applications using high level domain terms. This is basically the same process as used to construct domain models, as in Domain-Driven Design (DDD). Be aware that typically every application has structural aspects (e.g. entities and values that are required) as well as behavioral aspects (e.g., processes that operate on the entities, invariants or rules). For instance, in Reddit, automations can be used to add specific behaviour to its forums (see Figure 3.1). Other applications, like Amazon and Youtube, usually provide at least some process for verification of user evaluations and comments.

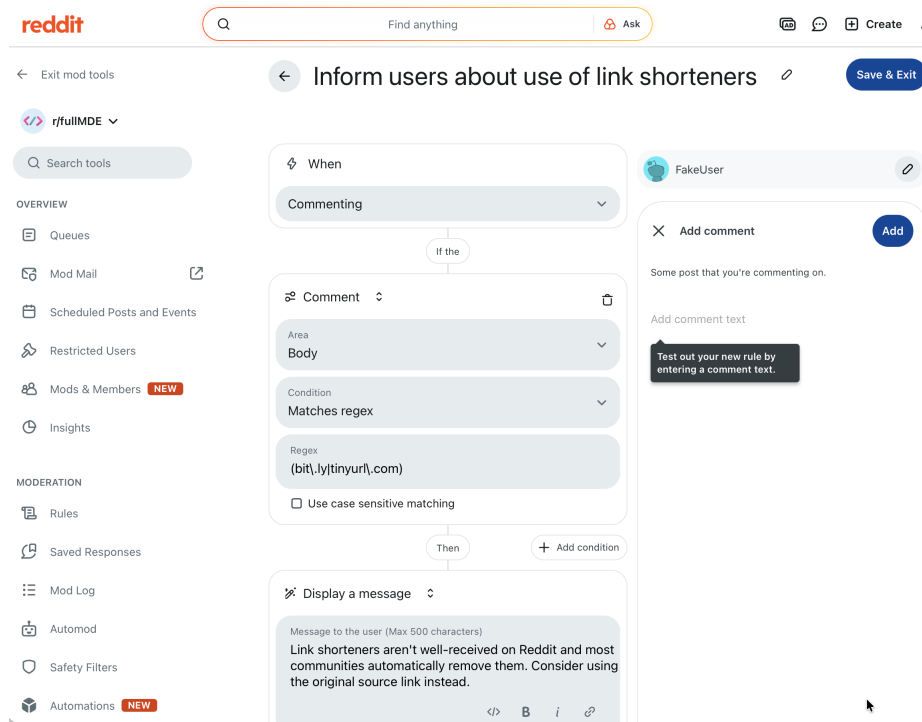
### 3.2 Metamodel

After completing the previous activity, which resulted in domain models for each scenario, the goal now is to design the metamodel for the new DSL.

The metamodel for the new DSL should:

- Result of the "union" of the previous domain models

Figure 3.1: Reddit Automations.



- This "union" must "merge" duplicated features
- The resulting metamodel should be concise but also complete regarding the identified features
- The resulting metamodel should also support all the presented aims of the project/DSL
- If properly justified, you may adapt the aims as well as add new features

### 3.3 Metamodel Constrains

Some constrains are easy to specify in class diagrams, such as, for instance, using the multiplicity in a relationship to constrain the number of referenced elements. However, some rules or constraints are difficult, or even impossible, to specify using only class diagram features. For instance:

- How to ensure that the values of a string field must always start with a capitalized letter?
- How to ensure that entity A can not depend on itself? Such as when A depends on B and B depends on A.
- etc.

For a model to be valid, all the domain rules must apply. Not only the model must be structurally correct but it also must meet all the other constraints. Only valid models should be used. For instance, only valid models should be used for the generation of code. This will ensure - assuming the generation process is correct - that the generated code will also be correct.

You should identify all the constraints for your metamodel and specify them in a formal way, for instance, using OCL (or at least using pseudo-code). If the tool you select for the implementation supports specifying constraints you should use that feature. If the tool has no direct support for that, you should implement the constraints in regular code and have a method (entry point) that can be executed to verify the correctness of a model.

### 3.4 Model Transformations

One of the fundamental concepts of Model-Driven Engineering is Model Transformations. Model transformations allow for generating target models (which also can be code) from source models. One of the uses of model transformations is model refactoring, i.e, a transformation where the source and target model are the same. Model refactoring can be used to propose model transformations that may correct issues or errors found by model constraints.

For each constraint, when suitable, you should propose a model refactoring that could correct the problem. For instance, if you have a constraint that verifies if an attribute starts with a capitalize letter you should design a model refactoring that capitalizes the first letter. Refactorings should be presented in a formal way or using pseudo-code. If your tool has no support for modeling refactorings you should implement them in regular code and have a method (entry point) that can be called to run refactorings over a model.

### 3.5 Model Visualizations/Projections

Full edition of models of the DSL is only a goal for P2. However, one of the concerns of the first part of the project, is providing support for the automatic generation of model visualizations or projections. A projection is a representation of a model, or part of a model, at a given moment. It is like a picture, it represents a model (or part of a model) at a given moment, and it is read-only, non-editable. There should be two types of projections: textual and graphical. The graphical projection should be a diagram that provides (in this case) a complete graphical representation of the model. The textual projection, as it names implies, is a textual representation of the model. Students should propose specific graphical and textual representations for each metamodel element. The specification of these representations should be included in the report.

For the graphical projection you should use the PlantUML (Graphviz) tool to generate the diagrams. For the textual representation the simple output of the text is sufficient.

Basically, each student should research an API for his/her tool to support the handling of models in a GPL (e.g., Java or C#) and generate the projection by visiting the nodes in the model and generating the necessary output for the representation of the node. If your tool supports generating "text/code" you can use that feature (i.e., and not use a more generic API). Figure 3.1 presents an excerpt of a possible textual projection for a REF model (this is only an example, your syntax may be significantly different).

Listing 3.1: Excerpt of possible textual projection for a REF model

```
USER TYPES : user, buyer IS user, seller
RESOURCE TYPE product : name AS TEXT, description AS TEXT, image AS IMAGE,
    WRITE seller, READ client
FEEDBACK-TYPE rateType : text AS TEXT, rate AS ONETO FIVE
FEEDBACK productReview : rateType, product, WRITE buyer, VALIDATION :
    automatic: automaticProductReviewVerification(buyer, text, rate)
FEEDBACK TYPE commentType : text AS TEXT
FEEDBACK comment : commentType, recursive, WRITE user, VALIDATION :
    automatic: automaticVerification(user, text)
...
```

For graphical projections, the student can generate the model diagram by "navigating" the model and producing a PlantUML textual file that represents the diagram. You may use the Dot language (supported by PlantUML, see <https://graphviz.org/doc/info/lang.html>) to be able to generate specific graphical representations. Listing 3.2 presents the source code of Plantuml that produces the graphical representation presented in Figure 3.2. **Note that this example is not related to the problem domain of the project, it is a simple illustration on how to use plantuml/dot to generate graphical projections**

Listing 3.2: Example of Plantuml/dot source code for generating a diagram

```
@startuml
digraph foo {
    start [shape=circle]
    end [shape=circle]

    init [shape=box, label="Init:\n Index=0 \n Max=LENGTH(Files)"]
    cycle [shape=box, label="Cycle:\n Index=Index+1"]
    testCycle [shape=diamond, label="TestCycle: \n Index<=Max"]
    interaction [shape=box, label="Interaction: \n Result=DEL(Files[Index])"]
    testDel [shape=diamond, label="TestDel: \n Result<0"]
    deleted [shape=box, label="Deleted: \n PRINT('File Deleted')"]
    error [shape=box, label="Error: \n PRINT('Unable to Delete File')"]

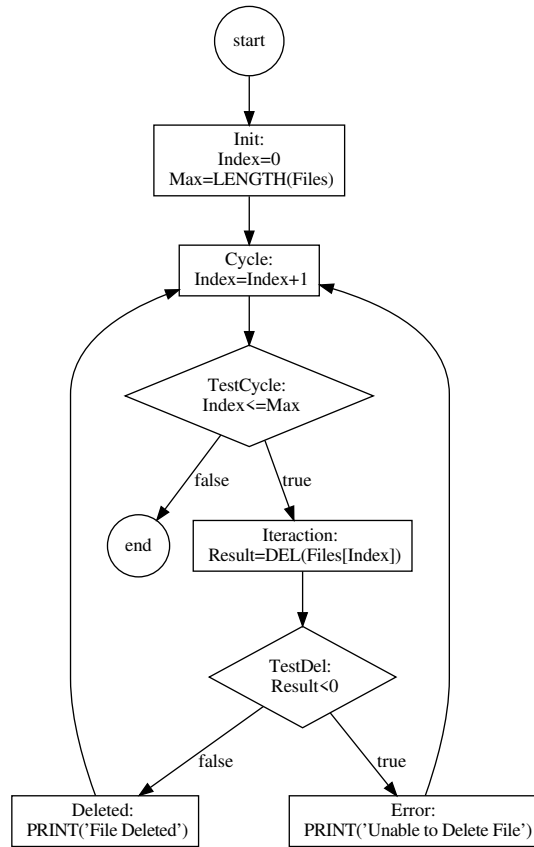
    start -> init
    init -> cycle
    cycle -> testCycle
    testCycle -> interaction [ label="true"]
    testCycle -> end [ label="false"]
    interaction -> testDel
    testDel -> deleted [ label="false"]
    testDel -> error [ label="true"]
}
```

```

deleted -> cycle
error -> cycle
}
@enduml

```

Figure 3.2: An example diagram generated by Plantuml/dot





## TERMS AND CONDITIONS

### 4.1 Submission

All the developed artifacts of the project should be included in your team repository (according to instructions).

For the Moodle submission a file containing a copy of the repository must be download from Github and submitted in Moodle using a specific link (of Moodle, that will be provided later).

### 4.2 Report

All the documentation should be included in the repository, using Markdown, and specific instructions included in the repository.

### 4.3 Deadline

**Until 11:59pm of April, 18th**

### 4.4 Feedback

Feedback can be request to the teacher at any moment, during the PL classes.

### 4.5 Evaluation

Evaluation for this part of the project will occur at the end of the semester, usually during lab classes, during weeks 13 to 15.

Each Student must be prepared to explain, discuss, operate and demonstrate his/her individual tasks and goals as well as the team's tasks and goals. Further instructions on evaluation sessions will be provided.

However, students may receive formative feedback during weeks 8 and 9, in the PL classes.

## 4.6 Grading

The weight of each part of the project will be provided in another document.

The evaluation of this project component has 2 parts:

- **Individual (60%).** This is specific to each team member and refers to his/her individual tasks and goals, in accordance to the performance of the student and his/her knowledge of the topics;
- **Team (40%).** This grade is related to the team's tasks and goals. It may be different for each team member, in accordance to the performance of the student and his/her knowledge of the topics;

### 4.6.1 Grading Criteria

A qualitative performance scale can be used by the students so that they can be aware of their own performance. This scale will also be used by teacher to grade each goal or task (described in Section 2.4)

Each task/goal can be graded following the next scale and the justification for each level:

- **0- nothing.** No submission or irrelevant;
- **1- tentative.** Do not achieve any of the requirements;
- **2- insufficient.** Achieves only a subset of the requirements, demonstrates insufficient knowledge of the evaluation subjects;
- **3- acceptable.** Achieves the requirements but with major faults, demonstrates acceptable knowledge of the evaluation subjects;
- **4- good.** Achieves the requirements with minor faults, justifies options, demonstrates good knowledge of the evaluation subjects;
- **5- excellent.** Achieves completely the requirements, justifies options, analyses alternatives in a quantified and rigorous way, demonstrates outstanding knowledge of the evaluation subjects;



The final grade for the project component results from the combination of the grades for all the tasks/goals (individual and team) and then converted to a 0-20 scale. Although in the scale only discrete values are presented (to facilitate the self-assessment by the student and also the feedback), teachers may use all the possible numeric values in the range of the scale 0-5.

## 4.7 Code of Conduct

- **Rules** cf. "Regulamento Disciplinar dos Estudantes do IPP"
- **Clearly identify sources and authorship** Code or any other material from external sources must be clearly identified in the code itself and in the reports, indicating the author or source.
- Any **misappropriation of authorship** (or collusion in the developed work) **will be sanctioned** according to the regulation, e.g., the evaluation component will be canceled. Also, the incident will be reported to the President of ISEP.
- **IA Tools usage** It is acceptable the usage of IA tools as a "research tool". However, it is expected (and mandatory) that all submitted artifacts are the student's authorship.